

AI활용프로그래밍

Week 4. 조건문과 반복문

상황에 따라 다르게 · 여러 번 되풀이하기

학습 목표

- 상황에 따라 다르게 동작하는 코드를 쓸 수 있다.
- 같은 일을 여러 번 되풀이하게 할 수 있다.
- 들여쓰기가 무엇을 묶는지 설명할 수 있다.
- 멈추지 않는 반복을 알아보고 고칠 수 있다.

오늘의 구성

- 1) 조건에 따라 갈라지는 코드
- 2) 들여쓰기가 정하는 묶음
- 3) 정해진 횟수만큼 되풀이하기
- 4) 조건이 맞는 동안 되풀이하기
- 5) With AI — 틀린 코드 고치기 실습 3개

조건문이 필요한 이유

- 지금까지 코드는 위에서 아래로 한 줄씩 실행됐습니다.
- 그런데 현실은 상황에 따라 답이 달라집니다.
- 점수가 90점이면 A, 아니면 다른 등급입니다.
- 이 갈림길을 만드는 것이 조건문입니다.

조건이 맞을 때만 실행하기

```
score = 95
if score >= 90:
    print("A")
```

실행 결과

A

핵심 포인트

- ▶ if 뒤의 조건이 맞으면 안쪽 줄을 실행합니다.
- ▶ 조건 줄 끝에는 콜론(:)을 붙입니다.
- ▶ 95는 90보다 크므로 A가 나옵니다.

용어 노트

조건문(if) — '만약 ~라면'으로 길이 갈리는 문장. 신호등 앞에서 갈지 멈출지 정하는 것과 같습니다.
조건 줄 끝 콜론(:) — 나는 문장이 끝나지 않았어요. 나에게는 따라오는 종속된 문장이 있습니다.

들여쓰기가 묶음을 정합니다

- 조건 줄 다음에 오는 안쪽 줄들이 한 묶음입니다.
- 묶음은 앞에 빈칸 네 칸을 넣어 표시합니다.
- 빈칸을 빼먹으면 오류가 납니다.
- 빈칸 개수가 들쭉날쭉해도 오류가 납니다.



용어 노트

들여쓰기(indent) — 줄 앞의 빈칸. 파이썬은 이것으로 어디까지가 한 묶음인지 판단합니다.

아니면 이렇게 (else)

```
score = 75
if score >= 90:
    print("A")
else:
    print("B")
```

실행 결과

B

핵심 포인트

- ▶ else는 '위 조건이 아닐 때'입니다.
- ▶ else 뒤에는 조건을 쓰지 않습니다.
- ▶ 둘 중 하나만 실행됩니다.

갈래가 셋일 때 (elif)

```
score = 85
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("C")
```

실행 결과

B

핵심 포인트

- ▶ elif는 '앞이 아니면 이걸 어때?'입니다.
- ▶ 위에서부터 차례로 확인합니다.
- ▶ 맞는 것 하나를 만나면 거기서 끝냅니다.
- ▶ elif는 여러 번 쓸 수 있습니다.

조건을 쓸 때 조심할 것

- `=`는 '넣어라', `==`는 '같은가?'입니다.
- 순서가 중요합니다. 큰 범위를 먼저 쓰면 안 됩니다.
- 90점이 C로 나오면 순서를 의심하세요.
- $0 \leq x \leq 100$ 처럼 범위도 한 번에 쓸 수 있습니다.

반복문이 필요한 이유

- 같은 줄을 백 번 복사해 붙일 수는 없습니다.
- 되풀이할 규칙만 적으면 컴퓨터가 알아서 반복합니다.
- 횟수가 정해졌으면 for를 씁니다.
- 조건이 맞는 동안이면 while을 씁니다.



용어 노트

반복문(for/while) — 같은 일을 여러 번 되풀이하는 문장. 노래 후렴을 반복해 부르는 것과 같습니다.

정해진 횟수만큼 (for)

```
for i in range(3):  
    print(i)
```

실행 결과

0

1

2

핵심 포인트

- ▶ range(3)은 0, 1, 2 세 개를 만듭니다.
- ▶ i에 그 값이 하나씩 들어갑니다.
- ▶ 안쪽 줄이 세 번 실행됩니다.

range 읽는 법

- `range(5)` — 0부터 4까지 다섯 개입니다.
- 0부터 시작하고, 끝 숫자는 포함하지 않습니다.
- `range(1, 4)` — 1부터 3까지 세 개입니다.
- 끝 숫자가 빠지는 것만 기억하면 됩니다.

1부터 세고 싶을 때

```
for i in range(1, 4):  
    print(i)
```

실행 결과

1
2
3

핵심 포인트

- ▶ 시작 숫자를 앞에 적습니다.
- ▶ 끝 숫자 4는 나오지 않습니다.
- ▶ 1부터 3까지 세 번 실행됩니다.

조건이 맞는 동안 (while)

- 몇 번 반복할지 미리 모를 때 씁니다.
- 조건이 맞는 동안 계속 되풀이합니다.
- 조건이 틀어지는 순간 반복을 끝냅니다.
- 그래서 안에서 조건을 바꿔 줘야 합니다.

세어 내려가기 (while)

```
n = 3
while n > 0:
    print(n)
    n = n - 1
```

실행 결과

3
2
1

핵심 포인트

- ▶ n 이 0보다 큰 동안 되풀이합니다.
- ▶ $n = n - 1$ 이 n 을 줄여 줍니다.
- ▶ n 이 0이 되면 조건이 틀어져 끝납니다.

멈추지 않는 반복을 조심하세요

- 안쪽에서 조건을 바꾸지 않으면 영원히 돕니다.
- 코랩에서는 칸 왼쪽 ■ 중지 단추로 멈춥니다.
- while을 쓸 때는 '무엇이 끝을 만드나'를 먼저 정합니다.
- 이것이 while에서 가장 흔한 실수입니다.



용어 노트

무한 루프(infinite loop) — 끝나지 않고 영원히 도는 반복. 끝내는 조건을 빠뜨리면 생깁니다.

차곡차곡 더하기

```
total = 0
for i in range(1, 6):
    total = total + i
print(total)
```

실행 결과

15

핵심 포인트

- ▶ 더할 값을 담을 이름을 0으로 시작합니다.
- ▶ 반복할 때마다 그 이름에 더해 줍니다.
- ▶ $1+2+3+4+5 = 15$ 가 나옵니다.
- ▶ '처음 값 → 반복하며 갱신'이 기본 골입니다.

이번 주 흔한 실수 다섯 가지

- 1) 안쪽 줄의 들여쓰기를 빠뜨림
- 2) 조건 줄 끝에 콜론(:)을 빠뜨림
- 3) 조건 순서를 거꾸로 씀
- 4) range의 끝 숫자가 포함된다고 착각
- 5) while 안에서 조건을 바꾸지 않음

With AI: 오늘은 '고치는 용도'로만

- 이번 학기 전반부(1~7주) 규칙입니다.
- 오늘은 오류 메시지가 없는 버그도 다룹니다.
- 그럴 때는 '기대한 답'과 '실제 답'을 같이 적습니다.
- 코드를 통째로 받아 붙이지 않습니다.



용어 노트

프롬프트(prompt) — AI에게 주는 질문이나 지시. 자세할수록 답이 좋아집니다.

프롬프트 템플릿 (기대 vs 실제)

나는 파이썬을 처음 배웁니다.
아래 코드가 원하는 대로
동작하지 않습니다.

[내 코드]
(코드 전체를 붙여넣기)

[기대한 답] 95점 → A

[실제 나온 답] 95점 → C

- 1) 왜 그런지 두 줄로 설명
- 2) 고칠 곳만 표시
- 3) 전체 코드는 주지 마세요

핵심 포인트

- ▶ 오류 메시지가 없을 때 쓰는 틀입니다.
- ▶ '기대한 답'과 '실제 답'을 나란히 적습니다.
- ▶ 이 두 줄만 있어도 원인 찾기가 쉬워집니다.
- ▶ 답을 받으면 내가 직접 고쳐 실행합니다.

실습 1. 들여쓰기가 빠진 코드

해야 할 것

- ▶ 아래 코드를 코드 칸에 넣고 실행합니다.
- ▶ 빨간 글씨의 마지막 줄을 읽습니다.
- ▶ 어느 줄에 빈칸이 빠졌는지 찾습니다.
- ▶ 고친 뒤 다시 실행합니다.

✓ 체크

실행하면 A 가 나옵니다.

실행 코드

```
score = 95
if score >= 90:
print("A")
```

[이런 오류가 납니다]

```
IndentationError:
expected an indented
block after 'if'
statement on line 2
```

[힌트]

indented 는
"들여쓰"이라는 뜻입니다.

실습 2. 순서가 뒤바뀐 조건

해야 할 것

- ▶ 코드를 실행합니다. 오류는 나지 않습니다.
- ▶ 95점인데 C가 나오는 것을 확인합니다.
- ▶ 왜 A로 가지 못했는지 생각합니다.
- ▶ 순서를 바꿔 다시 실행합니다.

✓ 체크

실행하면 A 가 나옵니다.

실행 코드

```
score = 95
if score >= 70:
    print("C")
elif score >= 90:
    print("A")
```

[나오는 답]

C

[힌트]

위에서부터 차례로
확인하고, 맞는 것
하나를 만나면
거기서 끝냅니다.

실습 3. 멈추지 않는 반복

해야 할 것

- ▶ 코드를 실행하면 멈추지 않습니다.
- ▶ 칸 왼쪽 ■ 중지 단추를 눌러 멈춥니다.
- ▶ 무엇이 n을 줄여 주지 않는지 찾습니다.
- ▶ 한 줄을 더해 3 2 1 만 나오게 합니다.

✓ 체크

3, 2, 1 을 출력하고 스스로 멈춥니다.

실행 코드

```
n = 3
while n > 0:
    print(n)
```

[무슨 일이 나나요]
3 이 끝없이 나옵니다.

[힌트]
n 이 계속 3 이라
조건이 들어지지
않습니다.

※ 반드시 ■ 중지를
눌러 멈추세요.

AI 답변 확인하는 법

- 고친 코드를 직접 실행해 봤는가?
- 경계에 있는 값도 넣어 봤는가? (89, 90, 91)
- 반복이 스스로 멈추는 것을 확인했는가?
- 고친 이유를 내 말로 말할 수 있는가?

미니 퀴즈 (5문항)

- 1) 안쪽 묶음을 정하는 것은? (콜론 / 들여쓰기)
- 2) range(1, 4)가 만드는 수는?
- 3) 90점이 C로 나오면 무엇을 의심할까요?
- 4) while이 안 멈추는 가장 흔한 이유는?
- 5) 같은지 묻는 기호는? (= / ==)

정답: 1-들여쓰기, 2-1,2,3, 3-조건 순서, 4-조건을 안 바꿈, 5-==

과제

- 필수 — 점수를 입력받아 등급을 출력하는 프로그램.
- A는 90점 이상, B는 80점대, 나머지는 C로 합니다.
- 95, 85, 60 세 값을 넣어 결과를 적어 둡니다.
- 제출은 코랩 공유 링크 또는 .ipynb 파일.
- 도전(선택) — 1부터 10까지 짝수만 출력하기.

오늘 정리 & 다음 주

- if로 갈림길을 만들고 elif로 갈래를 늘립니다.
- 묶음은 들여쓰기가 정합니다.
- 횟수가 정해졌으면 for, 조건이면 while입니다.
- while은 끝내는 조건을 반드시 넣습니다.
- 다음 주 — 자주 쓰는 코드에 이름을 붙여 재사용합니다.